

# Pi Agent 三层 Hook 系统架构

## ReAct Loop 时间线映射与事件驱动设计

技术架构白皮书 · 2026年6月

### 执行摘要

Pi Agent 采用独特的三层 Hook 系统架构，将事件驱动编程模型深度集成到 ReAct Loop 中。本文档系统地解析了三层 Hook——Agent Loop 层、Agent 事件层和 Harness/Extension 层——在整个 Agent 执行生命周期中的触发时机、设计原理与协作机制。通过类型安全的 HookEvent 系统和 Phantom Type 设计，Pi 实现了在 ReAct Loop 关键节点上的精细化控制能力。

## 一、三层 Hook 定义

| 层级           | 所在文件          | 核心特征             | 作用                   |
|--------------|---------------|------------------|----------------------|
| Agent Loop 层 | agent-loop.ts | 函数回调，直接参与执行逻辑    | 控制工具调用、转换消息格式、决定循环终止 |
| Agent 事件层    | agent.ts      | 只读事件流 AgentEvent | 状态同步、UI 更新、日志记录      |

|                         |                 |                        |                                 |
|-------------------------|-----------------|------------------------|---------------------------------|
| Harness/<br>Extension 层 | harness/ 目<br>录 | 类型安全的拦截系统<br>HookEvent | 支持 block/patch/cancel, 支持扩<br>展 |
|-------------------------|-----------------|------------------------|---------------------------------|

## 二、Agent Loop 层回调函数

| 回调函数                | 触发时机         | 返回值语义                     |
|---------------------|--------------|---------------------------|
| transformContext    | 消息进入 LLM 前   | 修改后的 AgentMessage[]       |
| convertToLlm        | 序列化为 LLM 请求前 | Message[] 格式转换            |
| beforeToolCall      | 工具调用执行前      | { block?: boolean } 可阻断   |
| afterToolCall       | 工具执行完成后      | { patch?: object } 可修改结果  |
| prepareNextTurn     | 每轮结束后        | AgentLoopTurnUpdate 修改上下文 |
| shouldStopAfterTurn | 停止决策点        | boolean 是否终止循环            |
| getApiKey           | Provider 请求前 | 认证密钥                      |

## 三、Agent 事件层事件类型

```
export type AgentEvent =
  | { type: "agent_start" }
  | { type: "agent_end"; messages: AgentMessage[] }
  | { type: "turn_start" }
  | { type: "turn_end"; message: AgentMessage; toolResults: ToolResultMessage[] }
  | { type: "message_start"; message: AgentMessage }
  | { type: "message_update"; message: AgentMessage; ... }
  | { type: "message_end"; message: AgentMessage } // → 触发 Session 写入
  | { type: "tool_execution_start/update/end"; ... }
```

## 四、Harness 层 Phantom Type 设计

```
declare const HookResult: unique symbol;

interface HookEvent<TType extends string, TResult = void> {
  type: TType;
  readonly [HookResult]?: TResult; // 只在类型层面存在
}
```

```
type ResultOf<E> = E extends { readonly [HookResult]?: infer R } ? R : void;
```

## 五、完整 ReAct Loop 时间线

### 1. 整轮循环开始

agent\_start   before\_agent\_start

### 2. 单轮开始 + Prompt 注入

turn\_start   message\_start/end   transformContext   context

### 3. LLM 请求准备

convertToLlm   before\_provider\_request   before\_provider\_payload   getApiKey

### 4. 流式响应 (Reason Phase)

message\_start   message\_update × N   after\_provider\_response   message\_end  
session.appendMessage()

### 5. 工具调用 (Act Phase)

beforeToolCall   tool\_call   tool\_execution\_start/update/end   afterToolCall   tool\_result  
message\_start/end

### 6. 回合结束 + 状态固化

turn\_end   prepareNextTurn   shouldStopAfterTurn   save\_point

### 7. 整轮结束

agent\_end   settled

## 六、AppendOnlyLog 写入机制

### 6.1 写入时机：双重策略

| 时机    | 触发事件           | 写入方式 | 说明                                                            |
|-------|----------------|------|---------------------------------------------------------------|
| 消息完成时 | message_end    | 立即写入 | 每完成一条消息(user/assistant/toolResult)，立即 session.appendMessage() |
| 回合结束时 | turn_end       | 批量刷新 | flushPendingSessionWrites() — 写入队列中的 model/thinking/tools 变更  |
| 空闲期操作 | phase = "idle" | 立即写入 | compaction、branch、model切换等结构性操作立即持久化                          |

### 6.2 三种消息的写入时机与数据结构

| 消息类型                                       | 触发时机                          | 修改窗口                                                     | 写入内容                                  | 数据结构                                                                                                                                                                                                                   |
|--------------------------------------------|-------------------------------|----------------------------------------------------------|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Prompt 消息</b><br>role: "user"           | agent_start 后<br>turn_start 时 | 无修改窗口<br>message_start<br>→ message_end<br>连续触发          | 原始用户输入                                | <pre>UserMessage {   role: "user";   content: string   (TextContent   ...);   timestamp: number; }</pre>                                                                                                               |
| <b>Assistant 消息</b><br>role: "assistant"   | LLM 流式响应完成后                   | 流式传输期间持续更新<br>message_update × N<br>流式结束后<br>message_end | 最终完整响应<br>(含 thinking/toolCalls)      | <pre>AssistantMessage {   role: "assistant";   content: (TextContent   ThinkingContent   ...);   api: Api;   provider: string;   model: string;   usage: Usage;   stopReason: StopReason;   timestamp: number; }</pre> |
| <b>ToolResult 消息</b><br>role: "toolResult" | 工具执行完成后                       | 无修改窗口<br>工具返回后立即生成                                       | 工具执行结果<br>(可被 tool_result hook patch) | <pre>ToolResultMessage {   role: "toolResult";   toolCallId: string;   toolName: string;   content: (TextContent   ImageContent   ...);   details?: any;   isError: boolean; }</pre>                                   |

```
        timestamp: number;
    }
}
```

关键设计原则：消息不可变性

一旦 `message_end` 触发，消息内容即视为最终确定，立即写入 `AppendOnlyLog`，之后不再修改。三种消息的差异在于：

- **Prompt 消息**：几乎瞬间完成，无修改机会，写入原始输入
- **Assistant 消息**：流式构建，在 `message_update` 期间内容持续变化，只有 `message_end` 时才最终定型并写入
- **ToolResult 消息**：工具同步返回，内容确定后立刻写入(但可在写入前通过 `tool_result hook` 修改)

6.3 官方消息修改入口

Pi 提供以下官方修改入口，在消息写入前对其进行修改：

| 修改目标                       | Hook / 回调                                                                    | 修改能力                                                                                       | 代码示例                                                                                                                                                                                                                                                                          |
|----------------------------|------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| System Prompt<br>系统指令      | <code>before_agent_start</code>                                              | 可修改<br><code>systemPrompt</code><br>可注入初始<br><code>messages</code>                         | <pre>harness.on("before_agent_start", async () =&gt; {     return {         systemPrompt: event.systemPrompt + " ",         messages: [{ role: "system", content: "你是一名助手" }];     }); });</pre>                                                                              |
| 进入 LLM<br>的消息列表<br>(裁剪/注入) | <code>context</code> 或<br><code>transformContext</code>                      | 可替换整个<br><code>messages</code> 数组<br>可添加/删除/修改任意消息                                         | <pre>harness.on("context", async (event) =&gt; {     // 1. 裁剪过长历史     const trimmed = event.messages.slice(-10);     // 2. 注入系统提示     const modified = [         ...trimmed,         { role: "user", content: "请记住：简洁" },     ];     return { messages: modified }; });</pre> |
| LLM 请求参数                   | <code>before_provider_request</code><br><code>before_provider_payload</code> | 可修改 <code>model</code> 、<br><code>streamOptions</code><br>可修改序列化后的<br><code>payload</code> | <pre>harness.on("before_provider_request", async () =&gt; {     return {         model: cheaperModel, // 降级模型         streamOptions: { ... }     }; });</pre>                                                                                                                 |
|                            | <code>tool_result hook</code>                                                |                                                                                            |                                                                                                                                                                                                                                                                               |

**ToolResult**  
内容

可 patch content  
可修改 details、  
isError

```
harness.on("tool_result", async (event) => {
  if (event.toolName === "read_file")
    return {
      content: "[截断]" + event.content,
      details: { truncated: true }
    };
});
```

下一轮的上  
下文

prepareNextTurn

可替换整个  
context.messages  
可切换 model/  
thinkingLevel

```
agent.prepareNextTurn = async (ctx) => {
  // 修改最后一条消息
  const modified = [...ctx.newMessages];
  modified[modified.length - 1].content = "[截断]";
  return {
    context: { ...ctx.context, message: modified },
    model: newModel
  };
};
```

## 关键限制

- **Prompt 消息**(已写入的): 无官方 API 修改, 设计上就是原始用户输入
- **Assistant 消息**(LLM 输出): 无官方 API 修改, 设计上保持 LLM 原始输出不可篡改
- 如需强制修改: 只能通过 prepareNextTurn 替换整条 context.messages 数组

## 对 Append Only Log 的友好性分析

不同修改入口对 Session 持久化的影响:

| 友好度         | 入口                                | 层级                        | Session 一致性                                                                  |
|-------------|-----------------------------------|---------------------------|------------------------------------------------------------------------------|
| ★★★★<br>最友好 | before_agent_start<br>tool_result | Harness                   | 修改后的内容正常走完 message_end → session.appendMessage(), <b>Session</b> 记录与实际执行完全一致 |
| ★★★<br>次友好  | context<br>prepareNextTurn        | Agent<br>Loop/<br>Harness | <b>Session</b> 存原始消息, 执行看修改后的消息——恢复时会得到与执行时不同的上下文, 存在一致性风险                   |
| ★<br>不友好    | before_provider_payload           | Harness                   | 完全绕过 <b>Session</b> 记录, Session 存的是原始 payload, 无法恢复实际发送的内容                   |

设计建议: 如需保证 Session 可恢复性与执行一致性, 优先使用 `before_agent_start` (注入初始消息)和 `tool_result` (修改工具返回), 避免使用 `context` 和 `before_provider_payload` 进行结构性修改。

## 6.4 Phase 管理与会话写入

```
type AgentHarnessPhase = "idle" | "turn" | "compaction" | "branch_summary" | "retry";

// "turn" 阶段: 活跃操作期
if (this.phase === "turn") {
  // 变更进入 pending 队列, 不立即写入
  this.pendingSessionWrites.push({ type: "model_change", ... });
}

// "idle" 阶段: 可立即写入
if (this.phase === "idle") {
  await this.session.appendModelChange(model.provider, model.id);
}
```

## 6.5 Session Entry 类型

所有状态变更建模为不可变的 Entry, 追加到 Session Tree:

```
type SessionTreeEntry =
  | MessageEntry           // 普通消息
  | ModelChangeEntry       // 模型切换
  | ThinkingLevelChangeEntry // thinking 级别变更
  | ActiveToolsChangeEntry  // 工具集变更
  | CompactionEntry        // 压缩操作
  | BranchSummaryEntry     // 分支摘要
  | LabelEntry             // 标签
  | CustomEntry;           // 自定义数据
```

# 七、状态恢复机制

## 7.1 恢复流程: 半持久化架构

### 1. Host App 重建 Runtime Dependencies

注册 tools    注册 models    注册 extensions/hooks    配置 auth providers

### 2. Harness 打开 Session



7.2 核心归约函数

```
export function buildSessionContext(pathEntries: SessionTreeEntry[]): SessionContext {
  // 从 entries 提取配置状态
  for (const entry of pathEntries) {
    if (entry.type === "thinking_level_change") {
      thinkingLevel = entry.thinkingLevel;
    } else if (entry.type === "model_change") {
      model = { provider: entry.provider, modelId: entry.modelId };
    } else if (entry.type === "active_tools_change") {
      activeToolNames = [...entry.activeToolNames];
    } else if (entry.type === "compaction") {
      compaction = entry; // 记录压缩点
    }
  }

  // 重建消息列表 (处理 compaction 截断)
  if (compaction) {
    // 压缩点之前的消息被 summary 替代
    messages.push(createCompactionSummaryMessage(...));
    // 只保留 firstKeptEntryId 之后的 entries
    for (const entry of keptEntries) appendMessage(entry);
  } else {
    for (const entry of pathEntries) appendMessage(entry);
  }

  return { messages, thinkingLevel, model, activeToolNames };
}
```

7.3 恢复策略

| 场景 | 默认策略 | 说明 |
|----|------|----|
|----|------|----|



|                  |                                                      |                                  |
|------------------|------------------------------------------------------|----------------------------------|
| 未完成的 Agent Turn  | 标记 <code>interrupted</code>                          | 保守策略, 不自动重试, 保留队列/pending writes |
| 未完成的 Provider 请求 | 标记 <code>interrupted</code>                          | Provider streams 不可恢复, 不自动重试     |
| 未完成的 Tool 调用     | 根据 metadata 决定                                       | 非幂等工具不重试, 幂等/可重试工具可重新执行          |
| 缺失的 Active Tools | <code>fail</code> (可配置 <code>drop / disable</code> ) | 工具 registry 必须提供兼容实现             |

7.4 设计哲学

半持久化架构: Session 存储可序列化的状态和配置(messages、model changes、tool changes), 但工具实现、模型对象、钩子逻辑等运行时依赖必须由 Host App 重新注入。

```
// 未来的恢复 Builder (设计中)
const harness = await AgentHarness.builder()
  .env(env)
  .session(session)           // 提供持久化 session
  .model(defaultModel)       // 提供默认模型
  .tools(runtimeTools)       // 提供运行时工具 (必须)
  .restore({ missingActiveTools: "fail" }); // 恢复策略
```

八、Pi Extension 扩展系统

8.1 Extension 与三层 Hook 的关系

Extension 是 Pi 的用户级扩展机制, 与三层 Hook 的关系如下:

| 层级                      | Extension 可用性 | 使用方式                                                                                   |
|-------------------------|---------------|----------------------------------------------------------------------------------------|
| Agent Loop 层            | ✗ 不可用         | 函数回调( <code>transformContext</code> , <code>prepareNextTurn</code> 等)仅供 Host App 内部使用  |
| Agent 事件层               | ✓ 可用(只读观察)    | 通过 <code>observe()</code> 订阅 <code>AgentEvent</code> , 用于 UI 更新、日志、监控                  |
| Harness/<br>Extension 层 | ✓ 主要使用        | 通过 <code>on()</code> 订阅 <code>HookEvent</code> , 支持 <code>block/patch/cancel</code> 语义 |

8.2 Extension 能操作的能力矩阵

| 能力类别                  | API / 事件                            | 操作范围                                |
|-----------------------|-------------------------------------|-------------------------------------|
| 消息修改<br>(Session 写入前) | before_agent_start                  | 修改 systemPrompt , 注入初始 message      |
|                       | context                             | 替换整个 messages 数组(裁剪/注入)             |
|                       | tool_result                         | patch content / details / isError   |
|                       | input                               | transform text/images , 或标记 handled |
| 注册能力                  | pi.registerTool()                   | 注册 LLM 可调用的工具(含自定义渲染)               |
|                       | pi.registerCommand()                | 注册用户命令(如 /my-cmd )                  |
|                       | pi.registerShortcut()               | 注册键盘快捷键(如 ctrl+k )                  |
|                       | pi.registerFlag()                   | 注册 CLI 参数(如 --my-flag )             |
| UI 交互<br>( ctx.ui )   | select/confirm/input                | 对话框交互(选择器、确认框、文本输入)                 |
|                       | setStatus/setWidget/setTheme        | 状态栏、组件、主题控制                         |
|                       | notify/editor/pasteToEditor         | 通知、多行编辑器、粘贴文本                       |
| Session 操作<br>( ctx ) | getContextUsage/<br>getSystemPrompt | 读取上下文用量、system prompt               |
|                       | newSession/fork/navigateTree        | 新建 session、分叉、树导航                   |
|                       | sendMessage/sendUserMessage         | 发送自定义消息、用户消息                        |

8.3 Extension 的完整事件订阅

```
type ExtensionEvent =
  // Session 生命周期
  | ResourcesDiscoverEvent
  | SessionStartEvent | SessionBeforeSwitchEvent | SessionBeforeForkEvent
  | SessionBeforeCompactEvent | SessionCompactEvent | SessionShutdownEvent
  | SessionBeforeTreeEvent | SessionTreeEvent
```

```
// Agent 执行（可修改的 HookEvent）
| ContextEvent                // 修改消息列表
| BeforeProviderRequestEvent  // 修改请求 payload
| BeforeAgentStartEvent       // 修改 systemPrompt
| ToolCallEvent               // 可 block, 可 mutate input
| ToolResultEvent             // 可 patch result

// Agent 执行（只读观察）
| AgentStartEvent | AgentEndEvent
| TurnStartEvent | TurnEndEvent
| MessageStartEvent | MessageUpdateEvent | MessageEndEvent
| ToolExecutionStartEvent | ToolExecutionUpdateEvent | ToolExecutionEndEvent
| AfterProviderResponseEvent

// 配置变更与用户输入
| ModelSelectEvent | ThinkingLevelSelectEvent
| UserBashEvent | InputEvent;
```

## 8.4 Extension 的限制

| 不能做的事                                                             | 原因                                                                       |
|-------------------------------------------------------------------|--------------------------------------------------------------------------|
| 修改已写入 Session 的 Prompt/Assistant 消息                               | 设计上不可变, AppendOnlyLog 一旦写入不可更改                                           |
| 直接使用 <code>transformContext</code> , <code>prepareNextTurn</code> | 属于 Agent Loop 层, 仅供 Host App 内部使用                                        |
| 访问原始 Agent 类实例                                                    | 通过 <code>ExtensionContext</code> 封装的安全 API 访问                            |
| 修改 <code>message_update</code> 期间的流式内容                            | <code>AssistantMessage</code> 流式期间不可修改, 只有 <code>message_end</code> 后可观察 |

## 8.5 Extension 设计哲学

沙盒化扩展模型: Extension 只能通过类型安全的 **Harness HookEvent** 参与 Agent 生命周期, 无法直接操作 Agent Loop 层的执行控制点。这种设计保证了:

- 稳定性: Extension 崩溃不会破坏核心 Agent 状态
- 可预测性: 所有修改通过声明式的事件结果对象传递
- 可审计性: Extension 的所有操作可追踪、可回放

一句话总结: **Extension = Harness 层事件订阅者 + 工具/命令/快捷键注册器 + UI 交互能力**。Extension 无法使用 Agent Loop 层的函数回调, 只能通过 Harness 层的类型安全事件系统参与 Agent 生命周期, 保证沙盒性和系统稳定性。

## 九、关键设计原则总结

**Agent Loop** 层 = "执行控制点" —— 同步调用, 直接返回值影响执行, 如 block 工具、修改消息格式、决定停止。

**Agent 事件层** = "状态通知流" —— 异步事件, 只读观察, 如 UI 更新、日志、监控。message\_end 触发 Session 写入, turn\_end 触发 pending writes 批量刷新。

**Harness/Extension** 层 = "拦截与扩展系统" —— 类型安全的 HookEvent, 支持 block/patch/cancel, 与 Agent 事件并行但独立。

**状态持久化** = "Append-Only Log" —— 每个消息和配置变更作为不可变 Entry 追加写入, 通过归约 (reduce) 重建当前状态, 实现崩溃恢复和会话重建。

一句话总结: Agent Loop 层控制"怎么执行", Agent 事件层通知"发生了什么"并驱动"何时写入", Harness 层决定"是否允许/如何修改", Session 层保证"状态可恢复" —— 四层交织形成完整的可信 Agent 架构。